
1 const

◆ Definition

The const keyword makes variables, pointers, function parameters, or member functions **read-only** after initialization.

◆ Why Important

- Prevents accidental modification of data.
- Enables **compiler optimizations** and **safer APIs**.
- Common in **finance systems** where state safety is critical (e.g., not modifying order IDs or timestamps).

◆ Types of const

1. **Constant variable**
2. `const int max_orders = 100;`
3. **Pointer to const / const pointer**
4. `const int* p = &x; // data is const`
5. `int* const q = &x; // pointer is const`
6. **Const member function**
7. `class Account {`
8. `int balance;`
9. `public:`
10. `int getBalance() const { return balance; } // cannot modify members`
11. `};`

Interview Tip

All `constexpr` are `const`, but not all `const` are `constexpr`.

2 static

◆ Definition

The static keyword controls **storage duration** and **visibility**.

◆ Types

1. **Static local variable** — persists between calls.
2. `void f() {`

3. `static int counter = 0;`
4. `counter++;`
5. `cout << counter;`
6. `}`
7. **Static class member** — shared among all objects.
8. `class Trade {`
9. `static int tradeCount;`
10. `public:`
11. `Trade() { tradeCount++; }`
12. `static void showCount() { cout << tradeCount; }`
13. `};`
14. `int Trade::tradeCount = 0;`
15. **Static global function/variable** — visible only within that file.

 Interview Tip

“Static = shared or persisted.”

Used in **low-latency counters, object pools, singletons.**

 3 auto

◆ **Definition**

Automatically **infers the type** at compile time from the initializer.

◆ **Example**

```
auto x = 3.14;    // double
```

```
auto y = 10;      // int
```

```
auto it = v.begin(); // STL iterator
```

◆ **Interview Use**

- Used heavily in **modern C++** with templates and lambdas.
- Reduces verbosity and helps generic programming.

 Interview Tip

Use auto when the type is obvious or very long (e.g., iterators).

 inline

◆ Definition

Suggests compiler to **replace function calls with the actual code** to remove call overhead.

◆ Example

```
inline int add(int a, int b) { return a + b; }
```

◆ Notes

- Only a **hint** — modern compilers decide automatically.
- Inline functions must be **defined in headers** if used in multiple files.

💬 Interview Tip

Good for **short, frequently called functions** in high-frequency paths.

5 virtual

◆ Definition

Used to enable **runtime polymorphism** — lets derived class override a base method.

◆ Example

```
class Base {  
public:  
    virtual void show() { cout << "Base"; }  
};  
  
class Derived : public Base {  
public:  
    void show() override { cout << "Derived"; }  
};
```

◆ Interview Tip

When function is virtual, C++ uses a **vtable lookup** at runtime — slightly slower but allows dynamic behavior.

Used in **trading engines, strategy selection, and event handling** frameworks.

6 override

◆ Definition

Introduced in C++11 — ensures a **derived class function truly overrides** a base virtual function.

◆ Example

```
class Base { virtual void log() const; };  
  
class Derived : public Base {  
    void log() const override { cout << "Logging..."; }  
};
```

💬 Interview Tip

Prevents hidden bugs when base/derived signatures mismatch.

7 macro

◆ Definition

Preprocessor directive (`#define`) that performs **text replacement** before compilation.

◆ Example

```
#define PI 3.14159  
  
#define SQUARE(x) ((x)*(x))
```

⚠ Pitfalls

- No type checking.
- Can cause debugging issues or side effects.

💬 Interview Tip

Modern C++ discourages macros — use `constexpr`, `inline`, or templates instead.

8 constexpr

◆ Definition

Guarantees **compile-time evaluation** of an expression.
Stronger form of `const`.

◆ Example

```
constexpr int square(int x) { return x * x; }  
  
constexpr int val = square(5);
```

◆ Why Important

- Ensures **zero runtime cost**.
- Used in **compile-time lookup tables**, **precomputed configs**, etc.

Interview Tip

All constexpr variables are const, but not vice versa.
Critical for **compile-time optimization** in HFT systems.

templates

◆ Definition

Allow writing **generic, type-independent** code.
Compiler generates code when instantiated.

◆ Example

```
template<typename T>
T add(T a, T b) { return a + b; }
```

◆ Advanced Forms

- Function templates
- Class templates
- Specialization (explicit/partial)
- Variadic templates

Interview Tip

Foundation of **generic programming, STL, and compile-time meta-programming**.
Used in **quant frameworks** for writing type-agnostic code.

10 Perfect Forwarding (std::forward)

◆ Definition

Allows forwarding of arguments **preserving their value category** (lvalue or rvalue).
Used in **factory patterns, resource management, and wrapper functions**.

◆ Example

```
void process(int& x) { cout << "Lvalue"; }
void process(int&& x) { cout << "Rvalue"; }
```

```
template<typename T>
void forwarder(T&& arg) {
    process(std::forward<T>(arg));
}
```

```
int main() {
    int x = 10;

    forwarder(x);    // Lvalue

    forwarder(20);   // Rvalue
}
```

Interview Tip

Common in **move semantics**, **template factories**, and **low-latency wrappers**.
Avoids **unnecessary copies** → faster.

1 Variadic Templates

◆ Definition

Templates that take **variable number of arguments** (like printf).
Introduced in C++11.

◆ Example

```
template<typename T>
void print(T arg) { cout << arg << " "; }

template<typename T, typename... Args>
void print(T first, Args... args) {
    cout << first << " ";
    print(args...);
}
```

```
int main() { print(1, 2.5, "Hi", 'A'); }
```

Interview Tip

Used in **logging**, **event dispatch**, **generic containers**, and **parameter packs** in HFT systems.

Summary Table

Concept	Category	Key Use	Example Use Case
const	Safety	Prevent modification	Immutable configs
static	Lifetime/Scope	Shared or persisted	Global counters
auto	Type deduction	Cleaner code	STL iterators
inline	Performance	Remove call overhead	Hot functions
virtual	OOP	Runtime polymorphism	Strategy switching
override	Safety	Ensure true override	OOP hierarchies
macro	Preprocessor	Text replacement	Constants/log macros
constexpr	Compile-time	Optimization	Precomputed constants
template	Generic programming	Type-agnostic code	STL
perfect forwarding	Efficiency	Forward args safely	Factory wrappers
variadic templates	Flexibility	Variable args	Logging/formatting